

Deep Learning

15 Self-Attention & Transformers - I

Motivation

- ① Why does one need to think beyond LSTMs?

Motivation

- ① Why does one need to think beyond LSTMs?
- ② Sequential processing doesn't allow parallelization
 - Path length = $\mathcal{O}(n)$
 - RNNs need $\mathcal{O}(n)$ steps to process a sentence of length n

Motivation

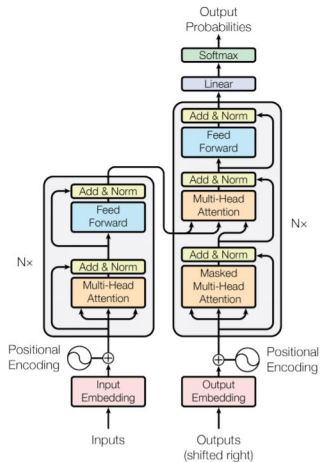
- ① (Despite the LSTM/GRU) RNNs need attention to deal with long-range dependencies

Motivation

- ① (Despite the LSTM/GRU) RNNs need attention to deal with long-range dependencies
- ② Since attention enables access to any state, do we need RNNs?

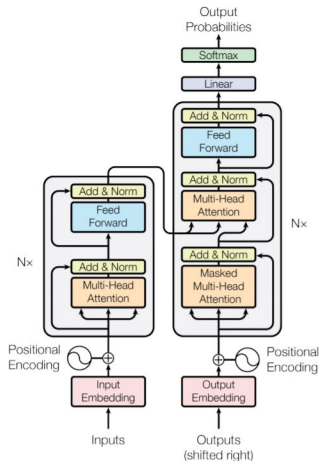
Transformers

- 1 Introduced by Vaswani et al.
NeurIPS 2017



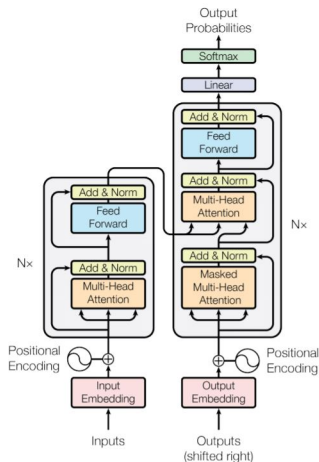
Transformers

- 1 Introduced by Vaswani et al. NeurIPS 2017
- 2 Sequence to sequence modeling without RNNs



Transformers

- 1 Introduced by Vaswani et al. NeurIPS 2017
- 2 Sequence to sequence modeling without RNNs
- 3 Transformer model is built on self-attention (no recurrence or convolutions)



Transformers

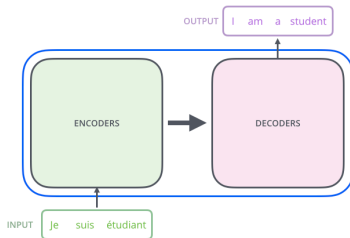


Credits: Jay Alammar

Transformers

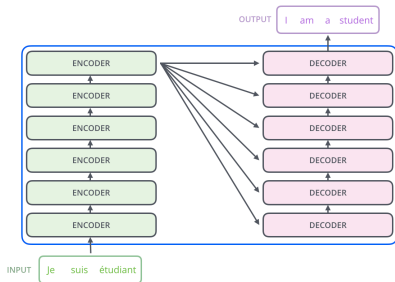


Credits: Jay Alammar



Credits: Jay Alammar

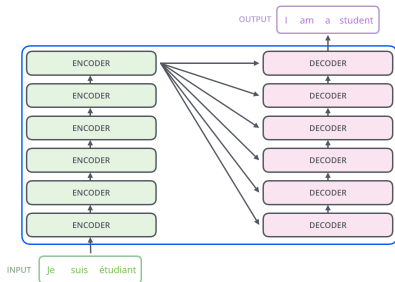
Transformers



Credits: Jay Alammar

- 1 The Encoding module has a stack of encoders

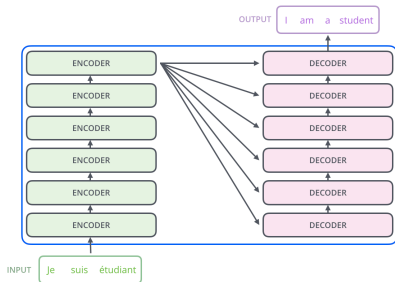
Transformers



Credits: Jay Alammar

- 1 The Encoding module has a stack of encoders
- 2 Same structure different parameters

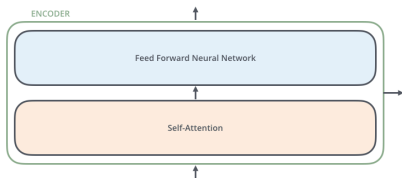
Transformers



Credits: Jay Alammar

- ① The Encoding module has a stack of encoders
- ② Same structure different parameters
- ③ Similarly, the decoding module

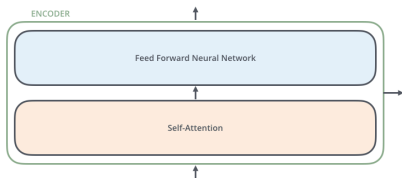
Transformers



- 1 Encoder first has a self-attention layer

Credits: Jay Alammar

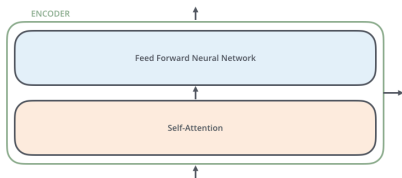
Transformers



Credits: Jay Alammar

- ① Encoder first has a self-attention layer
- ② Looks at the other words while encoding a specific word

Transformers



Credits: Jay Alammar

- ① Encoder first has a self-attention layer
- ② Looks at the other words while encoding a specific word
- ③ Next a (same) feed-forward NN is applied at all positions

Attention vs Self-attention

- ① Encoder-Decoder Attention
looks

Attention vs Self-attention

- ① Encoder-Decoder Attention looks
- ② **From:** a decoder (current) state

Attention vs Self-attention

- ① Encoder-Decoder Attention looks
- ② **From:** a decoder (current) state
- ③ **To:** all the encoder states

Attention vs Self-attention

① Encoder-Decoder Attention looks

② **From:** a decoder (current) state

③ **To:** all the encoder states

① Self-Attention looks

Attention vs Self-attention

- ① Encoder-Decoder Attention looks
- ② **From:** a decoder (current) state
- ③ **To:** all the encoder states

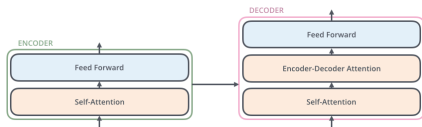
- ① Self-Attention looks
- ② **From:** each state from a set of states

Attention vs Self-attention

- ① Encoder-Decoder Attention looks
- ② **From:** a decoder (current) state
- ③ **To:** all the encoder states

- ① Self-Attention looks
- ② **From:** each state from a set of states
- ③ **To:** all other states in the same set

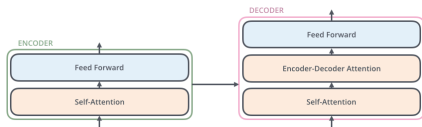
Transformers



Credits: Jay Alammar

- 1 Decoder has both the layers (self-attention and shared feed-forward)

Transformers



Credits: Jay Alammar

- 1 Decoder has both the layers (self-attention and shared feed-forward)
- 2 But, in the middle it has an **encoder-decoder attention layer**

Why the name 'Transformer'?

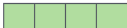
- ① Transforms a set of vectors in some representation space into a corresponding set of vectors (same dimensionality) in some new space

Why the name 'Transformer'?

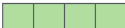
- ① Transforms a set of vectors in some representation space into a corresponding set of vectors (same dimensionality) in some new space
- ② **Goal:** new space will have a **richer internal representation** that is better suited to solve the downstream task

Transformers-Encoding

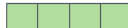
- ① Start with turning each word into a vector at the bottom-most encoder

x_1 

Je

x_2 

suis

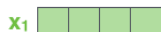
x_3 

étudiant

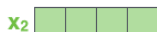
Credits: Jay Alammar

Transformers-Encoding

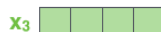
- ① Start with turning each word into a vector at the bottom-most encoder
- ② Others receive a list of vectors from the encoder immediately below



Je



suis



étudiant

Credits: Jay Alammar

Self-attention in Encoder vs. Decoder

- ① Who is doing: all source tokens

Self-attention in Encoder vs. Decoder

- ① Who is doing: all source tokens
- ② What are they doing (repeat)
 - look at each other
 - update representations

Self-attention in Encoder vs. Decoder

- | | |
|---|-----------|
| ① Who is doing: all source tokens | ① Decoder |
| ② What are they doing (repeat) | |
| • look at each other | |
| • update representations | |

Self-attention in Encoder vs. Decoder

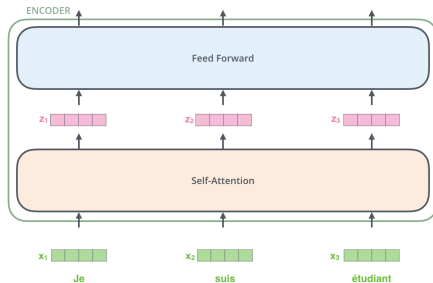
- | | |
|---|--|
| ① Who is doing: all source tokens | ① Decoder |
| ② What are they doing (repeat) <ul style="list-style-type: none">• look at each other• update representations | ② Who is doing: target token at each time step |

Self-attention in Encoder vs. Decoder

- ① Who is doing: all source tokens
 - ② What are they doing (**repeat**)
 - look at each other
 - update representations
- ① Decoder
 - ② Who is doing: target token at each time step
 - ③ What are they doing (**repeat**)
 - looks at previous target tokens (self-attention)
 - looks at source representations (encoder-decoder attention)
 - update representation

Transformers-Encoding

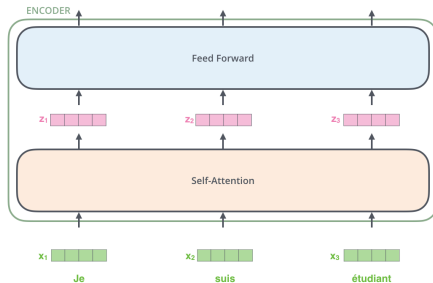
- ① Each word flows through the two layers of the encoder through its own path



Credits: Jay Alammar

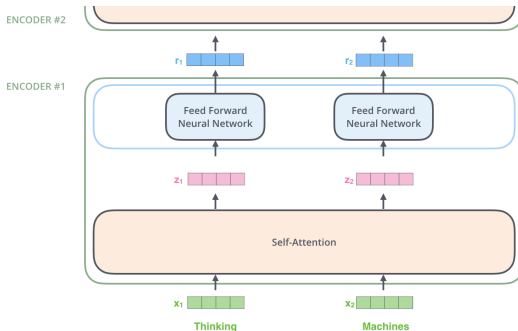
Transformers-Encoding

- ① Each word flows through the two layers of the encoder through its own path
- ② Self-attention layer has dependencies among them. However, the path length is $\mathcal{O}(1)$



Credits: Jay Alammar

Transformers-Encoding



Credits: Jay Alammarr

Self-Attention

- ① The animal didn't cross the street because it was too tired
- ② The animal didn't cross the street because it was too wide

Self-Attention

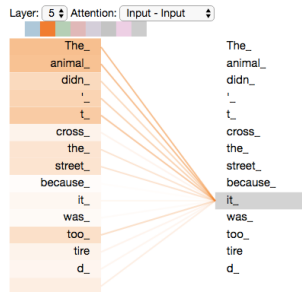
- ① The animal didn't cross the street because it was too tired
- ② The animal didn't cross the street because it was too wide
- ③ What does 'it' refer to?

Self-Attention

- ① The animal didn't cross the street because it was too tired
- ② The animal didn't cross the street because it was too wide
- ③ What does 'it' refer to?
- ④ Easy for humans, but not so much for the traditional Seq2Seq models

Self-Attention

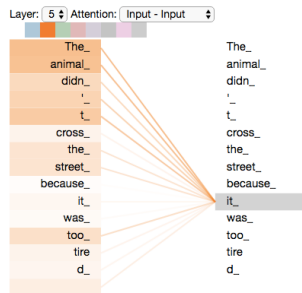
- 1 As the model processes each word, self-attention attends other positions in the i/p sequence to encode better



Credits: Jay Alammar

Self-Attention

- 1 As the model processes each word, self-attention attends other positions in the i/p sequence to encode better
- 2 Unlike RNNs, we don't keep hidden states from previous positions here!



Credits: Jay Alammar

Attention weights

- ① Input tokens $x_1, x_2, \dots, x_N$

Attention weights

- ① Input tokens $x_1, x_2, \dots, x_N$
- ② Output tokens $y_1, y_2, \dots, y_N$

Attention weights

- ① Input tokens $x_1, x_2, \dots, x_N$
- ② Output tokens $y_1, y_2, \dots, y_N$
- ③ $y_n = \sum_{m=1}^N a_{nm} \cdot x_m$

Attention weights

- ① Input tokens $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$
- ② Output tokens $\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_N$
- ③ $\mathbf{y}_n = \sum_{m=1}^N \mathbf{a}_{nm} \cdot \mathbf{x}_m$
- ④ $a_{mn} \geq 0$ and $\sum_{m=1}^N a_{mn} = 1$ Why?

How to compute the Attention weights?

- ① A simple way is to use the 'dot product' self-attention

How to compute the Attention weights?

① A simple way is to use the 'dot product' self-attention

②
$$a_{nm} = \frac{\exp(\mathbf{x}_n^T \mathbf{x}_m)}{\sum_{m'=1}^N \exp(\mathbf{x}_n^T \mathbf{x}_{m'})}$$

How to compute the Attention weights?

① A simple way is to use the 'dot product' self-attention

$$\textcircled{2} \quad a_{nm} = \frac{\exp(\mathbf{x}_n^T \mathbf{x}_m)}{\sum_{m'=1}^N \exp(\mathbf{x}_n^T \mathbf{x}_{m'})}$$

$$\textcircled{3} \quad \mathbf{Y} = \text{Softmax}[\mathbf{X}\mathbf{X}^T]\mathbf{X}$$

How to compute the Attention weights?

① A simple way is to use the 'dot product' self-attention

②
$$a_{nm} = \frac{\exp(\mathbf{x}_n^T \mathbf{x}_m)}{\sum_{m'=1}^N \exp(\mathbf{x}_n^T \mathbf{x}_{m'})}$$

③
$$\mathbf{Y} = \text{Softmax}[\mathbf{X}\mathbf{X}^T]\mathbf{X}$$

④ The transformation from $\mathbf{X}$ to $\mathbf{Y}$ is fixed; has no capacity to learn from the data

How to compute the Attention weights?

- ① A simple way is to use the 'dot product' self-attention
- ②
$$a_{nm} = \frac{\exp(\mathbf{x}_n^T \mathbf{x}_m)}{\sum_{m'=1}^N \exp(\mathbf{x}_n^T \mathbf{x}'_m)}$$
- ③ $\mathbf{Y} = \text{Softmax}[\mathbf{X}\mathbf{X}^T]\mathbf{X}$
- ④ The transformation from $\mathbf{X}$ to $\mathbf{Y}$ is fixed; has no capacity to learn from the data
- ⑤ Each of the feature values in a token plays an equal role in determining the attention weights

Some terminology from Information Retrieval

- ① Consider a streaming platform and the problem of choosing which movie to watch

Some terminology from Information Retrieval

- ① Consider a streaming platform and the problem of choosing which movie to watch
- ② One approach would be

Some terminology from Information Retrieval

- ① Consider a streaming platform and the problem of choosing which movie to watch
- ② One approach would be
- ③ Associate each movie with a list of attributes (genre, actors, technicians, length, year, etc.) → **Key**

Some terminology from Information Retrieval

- ① Consider a streaming platform and the problem of choosing which movie to watch
- ② One approach would be
- ③ Associate each movie with a list of attributes (genre, actors, technicians, length, year, etc.) → **Key**
- ④ These form a catalog of movies to be searched for

Some terminology from Information Retrieval

- ① Consider a streaming platform and the problem of choosing which movie to watch
- ② One approach would be
- ③ Associate each movie with a list of attributes (genre, actors, technicians, length, year, etc.) → **Key**
- ④ These form a catalog of movies to be searched for
- ⑤ The movie file itself is the **Value**

Some terminology from Information Retrieval

- ① Consider a streaming platform and the problem of choosing which movie to watch
- ② One approach would be
- ③ Associate each movie with a list of attributes (genre, actors, technicians, length, year, etc.) → **Key**
- ④ These form a catalog of movies to be searched for
- ⑤ The movie file itself is the **Value**
- ⑥ User's input of desired attributes → **Query**

Self-Attention

① $\mathbf{Q} = \mathbf{XW}^{(q)}$

Self-Attention

① $\mathbf{Q} = \mathbf{XW}^{(q)}$

② $\mathbf{K} = \mathbf{XW}^{(k)}$

Self-Attention

① $\mathbf{Q} = \mathbf{XW}^{(q)}$

② $\mathbf{K} = \mathbf{XW}^{(k)}$

③ $\mathbf{V} = \mathbf{XW}^{(v)}$

Self-Attention

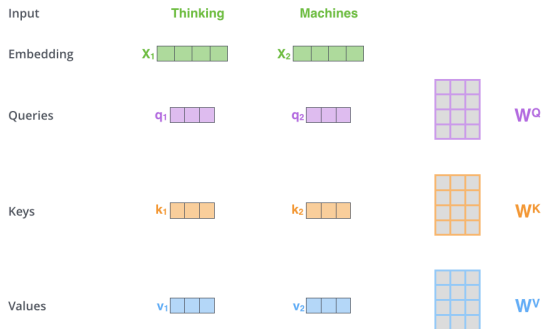
$$\textcircled{1} \quad \mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}$$

$$\textcircled{2} \quad \mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}$$

$$\textcircled{3} \quad \mathbf{V} = \mathbf{X}\mathbf{W}^{(v)}$$

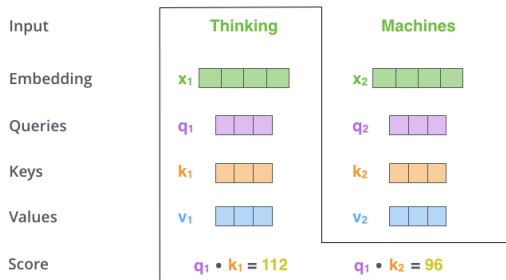
$$\textcircled{4} \quad \mathbf{Y} = \text{Softmax}[\mathbf{Q}\mathbf{K}^T]\mathbf{V}$$

Self-Attention



Credits: Jay Alammar

Self-Attention



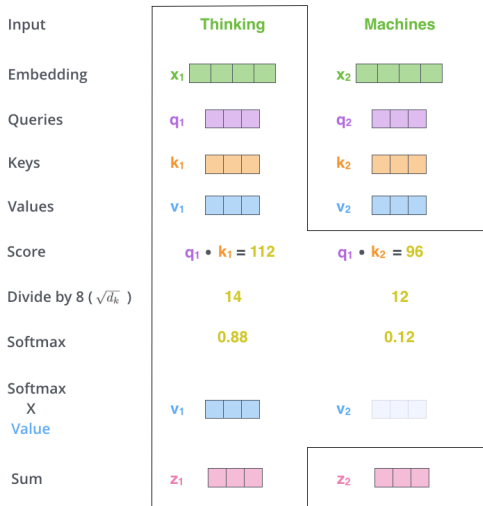
Credits: Jay Alammar

Self-Attention

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

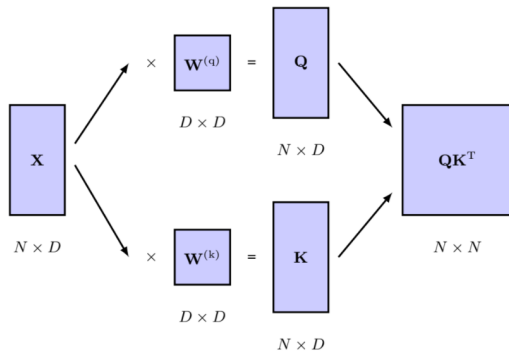
Credits: Jay Alammar

Self-Attention



Credits: Jay Alammar

Self-Attention



Credits: The Bishop's book

Self-Attention

The diagram illustrates the Self-Attention mechanism. On the left, a blue rectangle labeled Y represents the output, with dimensions $N \times D_v$ indicated below it. A small orange square in the top-left corner of Y highlights a specific element. This output is equal to the result of a Softmax operation applied to a matrix QK^T , which is shown inside a large curly brace. The matrix QK^T has dimensions $N \times N$ indicated below it. The top two rows of QK^T are highlighted in orange. To the right of the brace is a multiplication symbol $\times$, followed by a blue rectangle labeled V representing the input values, with dimensions $N \times D_v$ indicated below it. The first two columns of V are highlighted in orange, corresponding to the rows highlighted in QK^T .

$$Y_{N \times D_v} = \text{Softmax} \left\{ QK^T_{N \times N} \right\} \times V_{N \times D_v}$$

Credits: The Bishop's book

Scaled Self-Attention

- ① Gradients of the softmax become exponentially small for large input magnitudes

Scaled Self-Attention

- ① Gradients of the softmax become exponentially small for large input magnitudes
- ② To prevent this, the $\mathbf{QK}^T$ is scaled before the softmax

Scaled Self-Attention

- ① Gradients of the softmax become exponentially small for large input magnitudes
- ② To prevent this, the $\mathbf{QK}^T$ is scaled before the softmax
- ③ If the elements of q and v vectors are independent $N(0, 1)$ distributed, the variance of the dot product $\rightarrow D_k$

Scaled Self-Attention

- ① Gradients of the softmax become exponentially small for large input magnitudes
 - ② To prevent this, the $\mathbf{QK}^T$ is scaled before the softmax
 - ③ If the elements of q and v vectors are independent $N(0, 1)$ distributed, the variance of the dot product $\rightarrow D_k$
 - ④ Hence, normalize the product by the standard deviation
- $$\mathbf{Y} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left[\frac{\mathbf{QK}^T}{\sqrt{D_k}}\right]\mathbf{V}$$

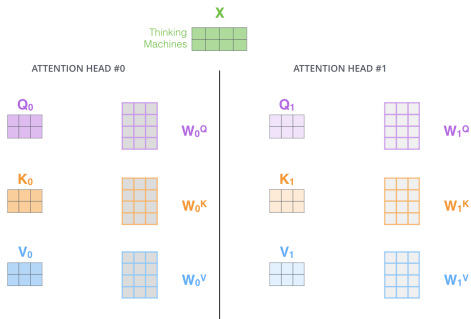
Multi-headed Self-Attention

- ① There may be multiple patterns of attention that are relevant at the same time

Multi-headed Self-Attention

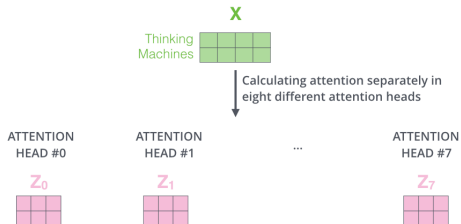
- ① There may be multiple patterns of attention that are relevant at the same time
- ② E.g., some patterns relevant to the 'tense' while others might be associated with the 'vocabulary.'

Multi-headed Self-Attention



Credits: Jay Alammar

Multi-headed Self-Attention



Credits: Jay Alammar

Multi-headed Self-Attention

- ① Expands the model's ability to focus on different relevant positions in the i/p

Multi-headed Self-Attention

- ① Expands the model's ability to focus on different relevant positions in the i/p
- ② Enables different 'representational subspace'

Multi-headed Self-Attention

1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

$\times$

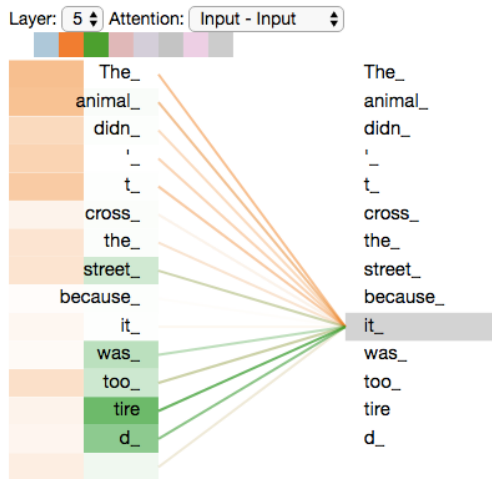


3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



Credits: Jay Alammarr

Multi-headed Self-Attention



Credits: Jay Alammar

Transformer Layers

- ① Neural nets benefit greatly from the depth

Transformer Layers

- ① Neural nets benefit greatly from the depth
- ② → stack multiple self-attention layers

Transformer Layers

- ① Neural nets benefit greatly from the depth
- ② → stack multiple self-attention layers
- ③ To improve the training efficiency, introduce residual connections (requires to maintain the dimensionality)

Transformer Layers

- ① Neural nets benefit greatly from the depth
- ② → stack multiple self-attention layers
- ③ To improve the training efficiency, introduce residual connections (requires to maintain the dimensionality)
- ④ Followed by Layer normalization
 $\mathbf{Z} = \text{LayerNorm}[\mathbf{Y}(\mathbf{X}) + \mathbf{X}]$

Transformer Layers

- ① Output vectors are constrained to lie in the subspace spanned by the i/p vectors

Transformer Layers

- ① Output vectors are constrained to lie in the subspace spanned by the i/p vectors
- ② Enhance the expressive capability/flexibility by post-processing using a nonlinear neural net (MLP)

Transformer Layers

- ① Output vectors are constrained to lie in the subspace spanned by the i/p vectors
- ② Enhance the expressive capability/flexibility by post-processing using a nonlinear neural net (MLP)
- ③ This should not affect the transformer's ability to process variable length i/p

Transformer Layers

- ① Output vectors are constrained to lie in the subspace spanned by the i/p vectors
- ② Enhance the expressive capability/flexibility by post-processing using a nonlinear neural net (MLP)
- ③ This should not affect the transformer's ability to process variable length i/p
- ④ Same share net applies to all the o/p tokens (followed by residual connection and normalization)

$$\tilde{\mathbf{X}} = \mathbf{LayerNorm}[\mathbf{MLP}[\mathbf{Z}] + \mathbf{Z}]$$